

Face Anti-Spoofing

Come funziona tutto il progetto — guida personale per capirlo

Progetto Biometric Systems

June 24, 2026

A cosa serve questo documento. Non è la relazione ufficiale da consegnare. È una spiegazione, passo per passo, di *tutto* quello che abbiamo costruito: cosa fa ogni pezzo, **perché** l'abbiamo fatto, e cosa significano i termini tecnici. L'obiettivo è che tu capisca la “baracca” a fondo, abbastanza da spiegarla a voce.

Contents

1	In una frase: cosa fa il sistema	2
2	Glossario: tutti gli acronimi	2
3	La pipeline a colpo d'occhio	3
4	Fase 1 — Setup dell'ambiente	4
5	Fase 2 — Il dataset	4
6	Fase 3 — Preprocessing e data augmentation	5
7	Fase 4 — Baseline classica: LBP + SVM	6
8	Fase 5 — Il modello deep: CNN fine-tuned	6
9	Fase 6 — Valutazione e confronto	7
10	Fase 7 — Cross-dataset: il test difficile (il pezzo “da voto alto”)	8
11	Fase 8 — Fusione dei due modelli (score-level)	9
12	Fase 9 — La demo	10
13	Fase 11 — Riconoscimento e sistema biometrico completo	11
14	Risultati in sintesi	12
15	Come spiegarlo all'orale (cheat-sheet)	12
A	Appendice A — Gli script (cosa fa ogni file)	13
B	Appendice B — I dati usati (filepath)	14

1 In una frase: cosa fa il sistema

Costruiamo un sistema che, data l'immagine di un volto, decide se è una **persona vera davanti alla camera** (*live*) oppure un **attacco** (*spoof*): una foto stampata, una foto mostrata su uno schermo, un video in replay. Questo problema si chiama **Face Anti-Spoofing** (o *Presentation Attack Detection*).

Perché serve — il problema

Un normale sistema di riconoscimento facciale (es. lo sblocco del telefono) confronta due volti. Ma se mostro alla camera una *foto* del proprietario, il sistema potrebbe sbloccarsi lo stesso: è stato “ingannato”. L'anti-spoofing è il modulo che sta *prima* del riconoscimento e blocca questi tentativi. È un classificatore binario: **live vs spoof**.

Abbiamo affrontato il problema in due modi diversi e poi li abbiamo combinati:

1. un metodo **classico** (estrazione manuale di feature di texture + classificatore **SVM**);
2. un metodo **deep learning** (una rete neurale **CNN** che impara le feature da sola);
3. la **fusione** dei due.

Abbiamo poi misurato quanto il sistema generalizza su un **secondo dataset mai visto** (la parte più interessante) e costruito una **demo** che funziona con la webcam. Infine (Fase 11) abbiamo aggiunto lo stadio di **riconoscimento del volto**: così il progetto non fa solo anti-spoofing ma diventa un **sistema biometrico completo** (anti-spoof + “chi sei?”).

2 Glossario: tutti gli acronimi

Tienilo a portata di mano: ogni termine ricompare nelle sezioni successive.

PAD / FAS

Presentation Attack Detection / Face Anti-Spoofing: il nostro task. Distinguere un volto reale (*bonafide*) da un attacco di presentazione.

live / spoof

Le due classi. **live** = persona vera (label 0). **spoof** = attacco: foto, schermo, stampa (label 1).

bonafide

Sinonimo “ufficiale” (standard ISO) di *live*: presentazione genuina.

CNN *Convolutional Neural Network*: rete neurale per immagini. Impara da sola quali pattern guardare, a differenza del metodo classico dove le feature le scegliamo noi.

ResNet-18

L'architettura di CNN che usiamo (18 livelli, con “collegamenti residui”). È pre-addestrata su ImageNet.

ImageNet

Enorme dataset di immagini generiche (1000 categorie). Le reti pre-addestrate su ImageNet partono già sapendo riconoscere bordi, texture, forme: ottimo punto di partenza.

SVM *Support Vector Machine*: classificatore classico. Cerca il confine che separa meglio le due classi. Lo usiamo con kernel **RBF**.

RBF *Radial Basis Function*: il “kernel” che permette all'SVM di tracciare confini curvi (non solo rette) tra le classi.

LBP *Local Binary Pattern*: descrittore di **texture**. Per ogni pixel guarda i vicini e produce un codice; l'istogramma di questi codici descrive la “grana” dell'immagine. Cuore del metodo classico.

YCbCr / HSV

Spazi colore alternativi all'RGB. Separano luminanza e cromaticità: gli artefatti dello spoof si vedono meglio nei canali di colore.

EER *Equal Error Rate*: la metrica principale. È il valore in cui i due tipi di errore (**FAR** e **FRR**) si eguagliano. **Più basso = meglio**.

FAR *False Acceptance Rate*: frazione di **spoof accettati come live** (l'errore “pericoloso”: un attacco passa).

FRR *False Rejection Rate*: frazione di **live rifiutati come spoof** (l'errore “fastidioso”: un utente vero viene bloccato).

ROC *Receiver Operating Characteristic*: curva che mostra il compromesso tra veri positivi e falsi positivi al variare della soglia.

AUC *Area Under the Curve* (della ROC): un numero in $[0, 1]$ che riassume la bontà del classificatore. **1 = perfetto**, 0,5 = a caso.

DET *Detection Error Tradeoff*: curva FAR vs FRR; un altro modo standard di visualizzare gli errori in biometria.

APCER / BPCER / ACER

Metriche dello standard **ISO/IEC 30107-3**. APCER = errore sugli attacchi (spoof presi per veri), BPCER = errore sui bonafide (veri presi per attacco), ACER = media dei due.

HTER

Half Total Error Rate = $(FAR + FRR)/2$. Usato nel cross-dataset, quando la soglia è **fissata in anticipo** e non ri-tarata.

AMP

Automatic Mixed Precision: durante il training usa numeri a 16 bit dove possibile $\Rightarrow$ più veloce e meno memoria GPU, senza perdere qualità.

GPU / VRAM

La scheda grafica (NVIDIA RTX 2060 SUPER, 8 GB di memoria) su cui giriamo il training della CNN.

OOM

Out Of Memory: errore di memoria esaurita. Lo abbiamo evitato (vedi Fase 2).

data augmentation

Trasformazioni casuali applicate alle immagini in training (ritaglio, flip, blur, compressione JPEG...) per rendere il modello più robusto.

3 La pipeline a colpo d'occhio

Il progetto è diviso in fasi, ognuna con una “verifica” (gate) prima di passare alla successiva. Schema:

Fase	Cosa produce
1. Setup	ambiente Python + GPU pronti
2. Dataset	146k immagini live/spoof su disco + indici CSV
3. Preprocessing	transforms e data augmentation
4. Baseline	SVM su feature LBP → <i>il riferimento da battere</i>
5. CNN	ResNet-18 fine-tuned → il modello forte
6. Valutazione	metriche + grafici di confronto
7. Cross-dataset	test su un dataset mai visto (il domain gap)
8. Fusione	combinazione SVM + CNN a livello di score
9. Demo	inferenza su immagine e webcam in tempo reale
11. Recognition	embedding + identificazione open-set → <i>sistema completo</i>

Perché questo step

Lavorare a fasi con un gate dopo ognuna evita di accorgersi alla fine che qualcosa, a monte, era sbagliato. Ogni fase produce un artefatto verificabile (un numero, un grafico, un file) prima di costruirci sopra la successiva. È lo stesso principio dei test in ingegneria del software.

4 Fase 1 — Setup dell'ambiente

Cosa. Abbiamo creato un ambiente Python isolato (`.venv`) e installato PyTorch (con supporto GPU CUDA), `timm` (le architetture pronte), OpenCV, scikit-learn, scikit-image. Verificato che la GPU sia vista da PyTorch.

Perché questo step

Una **virtualenv** tiene le librerie del progetto separate dal sistema: versioni riproducibili, niente conflitti. La verifica della GPU è fondamentale perché addestrare una CNN su CPU sarebbe lentissimo: sulla RTX 2060 un'epoca dura ~5 minuti, su CPU sarebbe ore.

Nota pratica: le dipendenze sono in `requirements.txt`. PyTorch va installato a parte dall'indice ufficiale (il pacchetto con CUDA non sta su PyPI).

5 Fase 2 — Il dataset

Cosa. Abbiamo usato **CelebA-Spoof**, uno dei più grandi dataset pubblici di anti-spoofing, nella versione *cropped* (volti già ritagliati) presa da Hugging Face. Scaricato un sottoinsieme (~12 GB di file `parquet`), poi **estratto in PNG su disco** con un indice `.csv` (colonne `filepath`, `label`).

Split	Immagini	Live	Spoof
TRAIN	79.183	33,2%	66,8%
→ train (fit)	67.306	33,0%	67,0%
→ val (early stop)	11.877	34,1%	65,9%
TEST (disgiunto)	66.787	29,7%	70,3%

Perché questo step

Perché un dataset grande e pubblico: servono decine di migliaia di esempi per addestrare una CNN senza che impari “a memoria”; ed essendo pubblico i risultati sono confrontabili con la letteratura.

Perché estrarre i PNG su disco: caricare tutti i `parquet` (con le immagini dentro) in RAM la saturava → errore **OOM**. Scrivendo i file su disco una volta sola, il training legge

ogni immagine “on-the-fly” con poca memoria.

Perché tre split (train/val/test): *train* = su cui il modello impara; *val* = per decidere quando fermarsi e tarare la soglia (**non** per imparare); *test* = il “compito d’esame”, visto solo alla fine per dare il voto onesto.

Concetto chiave: il “data leakage” e perché il test è separato

Se il modello vedesse in fase di test gli stessi dati (o lo stesso soggetto) su cui si è allenato, otterrebbe un voto gonfiato e bugiardo. Qui il **test viene dallo split ufficiale disgiunto** di CelebA-Spoof, quindi soggetti diversi: la valutazione finale è pulita. Il *val*, ricavato dal *train*, può condividere soggetti col *train*, ma va bene perché serve solo a fermare il training al momento giusto, non a dare il voto.

Nota sullo sbilanciamento. Ci sono circa il doppio di spoof rispetto a live ($\sim 2:1$). Se non facessimo nulla, il modello sarebbe “pigro” e tenderebbe a dire sempre “spoof”. Lo gestiamo nelle fasi successive con i *class weights* / sottocampionamento bilanciato.

6 Fase 3 — Preprocessing e data augmentation

Cosa. Poiché i volti sono già ritagliati, il preprocessing si riduce a: ridimensionare a 224×224 , normalizzare con le statistiche di ImageNet, e — solo in training — applicare **augmentation casuale**: ritaglio, flip orizzontale, variazioni di colore/luminosità, sfocatura, **ricompressione JPEG casuale**, cancellazione di una zona (random erasing).



Figure 1: Esempi di immagini dopo l’augmentation: ogni copia è leggermente diversa.

Perché questo step

Perché normalizzare con le statistiche di ImageNet: la CNN è pre-addestrata su ImageNet, quindi si aspetta input con quella scala di colori; rispettarla rende il fine-tuning più stabile.

Perché l’augmentation (e proprio queste trasformazioni): mostrando al modello molte varianti della stessa immagine gli impediamo di imparare dettagli irrilevanti (“overfitting”) e lo rendiamo **robusto**. In particolare blur e **JPEG casuale** imitano gli artefatti

di stampa e di replay: è l'augmentation che aiuterà di più nel cross-dataset (Fase 7).

7 Fase 4 — Baseline classica: LBP + SVM

Cosa. Prima di tirare fuori il deep learning costruiamo un **metodo di riferimento** semplice e classico, basato sull'articolo di Boulkenafet et al. (2015):

1. per ogni immagine calcoliamo istogrammi **LBP** (texture) su due spazi colore (**YCbCr** e **HSV**), su più scale, per ogni canale → un vettore di **168 numeri** che descrive la “grana” dell'immagine;
2. addestriamo un **SVM** con kernel **RBF** a separare live da spoof su questi vettori.

Perché la texture distingue live da spoof

Una foto reale e la *foto di una foto* hanno micro-texture diverse: lo schermo aggiunge pattern di pixel (effetto moiré), la stampa aggiunge grana e riflessi, la ricompressione aggiunge artefatti. L'**LBP** cattura proprio questi pattern locali, e sono più evidenti nei **canali di colore** (per questo YCbCr/HSV e non solo RGB).

Perché questo step

Perché fare una baseline prima della CNN: ti dà un **metro di paragone** onesto. Senza, non sapresti se l'EER della CNN è “buono”. La baseline risponde alla domanda “quanto è difficile questo problema con metodi semplici?”.

Perché un sottocampione bilanciato per l'SVM: l'SVM RBF non scala a decine di migliaia di esempi (diventa lentissimo) ed è sensibile allo sbilanciamento; addestriamo su 12.000 immagini bilanciate. La **valutazione** è però sul test completo (66.787 immagini).

Risultato: **EER = 7,69%**, **AUC = 0,976**. È il numero da battere.

8 Fase 5 — Il modello deep: CNN fine-tuned

Cosa. Prendiamo una **ResNet-18 pre-addestrata su ImageNet**, le sostituiamo l'ultimo strato con uno a 2 uscite (live/spoof) e la ri-addestriamo (*fine-tuning*) sul nostro dataset. Dettagli del training:

- **AMP** (mixed precision) per stare negli 8 GB di VRAM ed essere veloci;
- **class weights** (peso maggiore alla classe meno frequente) per lo sbilanciamento 2:1;
- **early stopping** sull'EER di validation: ci fermiamo quando smette di migliorare;
- scheduler *cosine* che riduce gradualmente il learning rate.

5 epoche, ~24 minuti totali sulla GPU.

Cos'è il “transfer learning” / fine-tuning

Invece di partire da una rete “a caso”, partiamo da una già addestrata a riconoscere immagini generiche (ImageNet). Sa già vedere bordi, texture, forme; noi le insegniamo solo la nuova distinzione live/spoof. Risultato: ottimi risultati con **poco tempo e pochi dati** rispetto ad addestrare da zero.

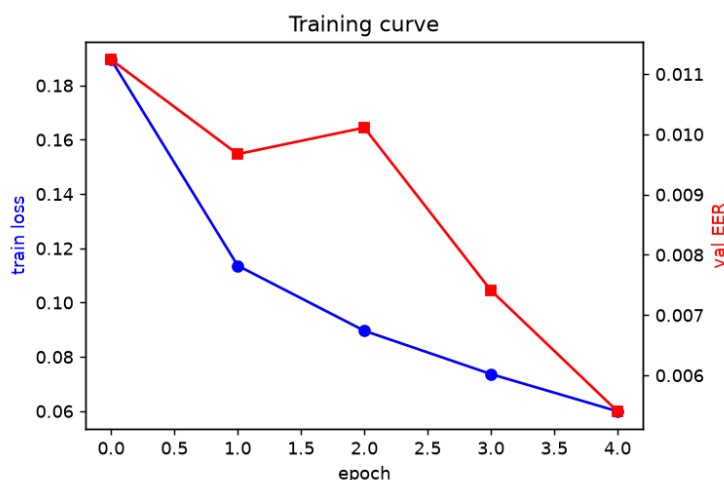


Figure 2: Curva di training: la loss (blu) scende, l'EER di validation (rosso) scende fino a $\sim 0,5\%$. Nessun overfitting (le due curve scendono insieme).

Perché questo step

Perché una CNN se la baseline già funziona: la CNN **impara da sola** quali dettagli guardare, anche quelli che noi non sapremmo descrivere a mano con l'LBP. Ci aspettiamo (e otteniamo) un netto miglioramento.

Perché early stopping sul val: continuare ad allenare troppo a lungo fa “imparare a memoria” il train (overfitting). Fermarci quando il val smette di migliorare ci dà il modello che generalizza meglio. **Perché i class weights:** per non far ignorare al modello la classe live (minoritaria).

Risultato: $\text{EER} = 3,69\%$, $\text{AUC} = 0,994$. La CNN dimezza l'errore della baseline ($7,69\% \rightarrow 3,69\%$).

9 Fase 6 — Valutazione e confronto

Cosa. Abbiamo un modulo unico che carica gli score salvati dei due modelli, ricalcola tutte le metriche e disegna le curve **ROC** e **DET sovrapposte**, più una tabella riassuntiva.

Modello (test, 66.787 img)	EER	AUC	ACER
Baseline SVM (color-LBP)	7,69%	0,976	7,69%
CNN (ResNet-18)	3,69%	0,994	3,69%

Perché questo step

Perché un framework di valutazione separato: così tutti i modelli vengono misurati con **esattamente le stesse metriche e lo stesso codice** $\rightarrow$ confronto equo. Abbiamo anche un “test di sanità”: dando al codice predizioni perfette deve dare EER 0, e predizioni casuali $\text{EER} \approx 0,5$. Entrambi verificati: ci fidiamo dei numeri.

Come leggere EER, ROC e perché “spoof = positivo”

Il modello dà a ogni immagine uno **score di “spoofness”** (alto = probabile attacco). Scegliendo una **soglia** decidiamo da dove in poi chiamarla spoof. Soglia alta $\rightarrow$ pochi falsi

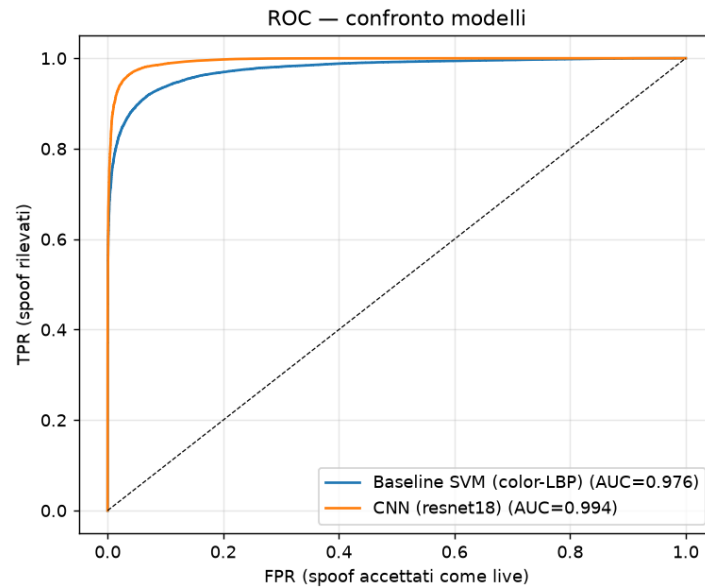


Figure 3: ROC a confronto: la curva della CNN “abbraccia” l’angolo in alto a sinistra (meglio). Più l’area sotto la curva (AUC) si avvicina a 1, migliore è il modello.

allarmi ma qualche attacco passa; soglia bassa → blocchi tutti gli attacchi ma anche utenti veri. La **ROC** disegna questo compromesso per tutte le soglie; l’**EER** è il punto di equilibrio ($FAR = FRR$). Trattiamo lo spoof come “positivo” (=l’evento da rilevare, come un attacco/un impostore): è la convenzione biometrica.

10 Fase 7 — Cross-dataset: il test difficile (il pezzo “da voto alto”)

Cosa. Un modello può sembrare perfetto sul suo dataset e crollare su immagini prese “nel mondo reale”, con altre camere, luci, attacchi. Per misurarlo abbiamo preso un **secondo dataset completamente diverso e mai visto in training: CASIA-FASD** (1655 volti). Protocollo:

- fissiamo la **soglia di decisione sul val di CelebA** (il dominio sorgente);
- applichiamo la **stessa soglia** sia su CelebA-test (*intra*, stesso dominio) sia su CASIA (*cross*, dominio nuovo) e misuriamo l’**HTER**.

Modello	Scenario	EER	AUC	HTER
CNN	INTRA (CelebA test)	3,69%	0,994	7,91%
CNN	CROSS (CASIA)	26,51%	0,832	35,36%
SVM	INTRA (CelebA test)	7,69%	0,976	8,50%
SVM	CROSS (CASIA)	34,43%	0,730	44,37%

Perché questo step

Perché fissare la soglia sul val sorgente e non ri-tararla sul target: nel mondo reale non conosci in anticipo gli attacchi futuri, quindi devi scegliere la soglia “a casa” e sperare che tenga. Ri-tararla sul dataset nuovo sarebbe “barare” (useresti dati che non avresti). L’**HTER** con soglia fissa misura quindi lo scenario realistico.

Perché questo è il pezzo da voto alto: mostra onestà scientifica. Tutti riportano i numeri belli “intra-dataset”; far vedere il **crollo cross-dataset** (l’HTER della CNN passa

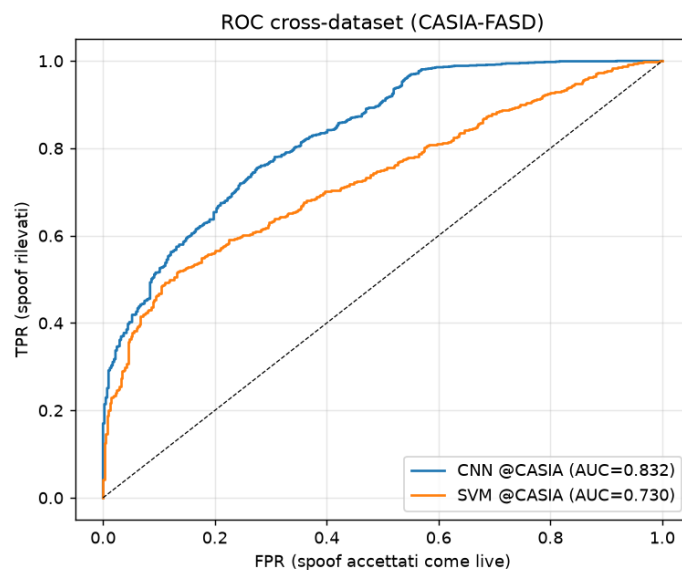


Figure 4: ROC sul dataset nuovo (CASIA): entrambi i modelli calano, ma la CNN (AUC 0,83) regge meglio della baseline (AUC 0,73).

da 7,9% a 35,4%) e **spiegarlo** dimostra che hai capito il vero problema della biometria: la **generalizzazione**.

Cos'è il “domain gap”

“Dominio” = la distribuzione dei dati (camera, luce, tipo di attacco, etnia...). Il **domain gap** è il calo di prestazioni quando il dominio di test è diverso da quello di training. È il limite pratico più serio dei sistemi anti-spoofing: un modello allenato su un dataset spesso non regge sugli attacchi “nuovi” di un altro. Conclusione del nostro esperimento: la **CNN generalizza meglio** della baseline (AUC 0,83 vs 0,73), ma il gap resta importante.

11 Fase 8 — Fusione dei due modelli (score-level)

Cosa. Combiniamo gli score di SVM e CNN per vedere se insieme fanno meglio del migliore da solo. Dato che i due score non sono confrontabili (l'SVM dà numeri illimitati, la CNN una probabilità in $[0, 1]$), prima li **normalizziamo** (min-max, parametri stimati sul val), poi proviamo varie regole di combinazione (somma, media, prodotto, max, min, somma pesata).

Metodo (test)	EER	AUC	ACER
CNN (solo)	3,69%	0,994	3,69%
Fusion sum/mean	3,35%	0,991	3,35%
Fusion product	3,50%	0,9948	3,50%
Fusion weighted (w=0,10)	5,14%	0,987	5,14%

Perché questo step

Perché fondere: due modelli che sbagliano su immagini diverse, combinati, si correggono a vicenda. È il **multi-biometria a livello di score** del corso (Cap. 11 del manuale).

Perché normalizzare prima: sommare un numero “-3.7” (SVM) con “0.8” (CNN) non ha senso finché non li riporti sulla stessa scala $[0, 1]$.

L’insight onesto: la fusione migliora poco (EER 3,69% $\rightarrow$ 3,35%) perché la CNN **domina già** la baseline. E la “somma pesata” con peso ottimizzato sul val **peggiora** sul test (5,14%): è **overfitting** del peso sul val. Morale: la regola semplice (somma a peso uguale) generalizza meglio di quella “furba”.

12 Fase 9 — La demo

Cosa. Una classe `SpoofDetector` che: trova il volto nel frame (face detector di OpenCV), lo ritaglia, lo passa alla CNN e restituisce **LIVE/SPOOF** + lo score. Tre script:

- `single_image.py`: una foto $\rightarrow$ verdetto + immagine annotata;
- `webcam_demo.py`: webcam in tempo reale, riquadro **verde REAL** / **rosso SPOOF**;
- `calibrate.py`: cattura qualche tuo frame e fa un mini fine-tuning per adattare il modello alla tua webcam/luce.



Figure 5: Esempio di output annotato della demo.

Perché questo step

Perché una demo: dimostra che il sistema non è solo numeri su un foglio ma **funziona davvero** in tempo reale — l’effetto che colpisce all’orale.

Perché la calibrazione: il modello è allenato su CelebA; la tua webcam è un “dominio nuovo” (vedi Fase 7!), quindi può sbagliare. Catturando qualche tuo frame live/spoof e ri-allenando 5 epoche, riduci il domain gap *sul tuo setup specifico*. È la stessa lezione della Fase 7 applicata in pratica.

13 Fase 11 — Riconoscimento e sistema biometrico completo

Perché questa fase. Tutto quello visto finora fa *anti-spoofing* (live vs spoof): risponde a “è reale?”, non a “chi sei?”. Un sistema biometrico vero fa anche **riconoscimento**. Qui aggiungiamo lo stadio mancante e otteniamo la pipeline completa:

volto → **anti-spoof** (live/spoof) → se LIVE → **riconoscimento** (chi sei? / sconosciuto)

I termini nuovi

Embedding: un vettore di 512 numeri che rappresenta un volto; foto della stessa persona → vettori vicini, persone diverse → vettori lontani. **Cosine similarity**: quanto sono “allineati” due embedding (1=identici, ~0=diversi). **Gallery**: l’archivio degli iscritti (un embedding per soggetto). **Probe**: la foto da identificare. **Verification (1:1)**: “sei davvero X?”. **Identification (1:N)**: “chi sei tra gli N iscritti?”. **Open-set**: può anche rispondere “nessuno”. **rank-1**: % di probe il cui match corretto è il primo della lista. **CMC**: curva che mostra il rank-1, rank-2... **MTCNN**: il rilevatore che trova e allinea il volto.

Come. Usiamo **facenet-pytorch**: **MTCNN** ritaglia/allinea il volto, **InceptionResnetV1** (pre-addestrata su **VGGFace2**, un grande dataset di volti) lo trasforma in un embedding da 512 numeri. Confronto tra volti = cosine similarity. La **soglia di accettazione** non è scelta a occhio: la calibriamo sul benchmark pubblico **LFW** (*Labeled Faces in the Wild*).

Perché questo step

Perché LFW e non CelebA-Spoof: il nostro dataset anti-spoofing (cropped) **non contiene le identità** dei soggetti, quindi non ci si può valutare il riconoscimento. LFW è il benchmark standard *con* identità etichettate → lo usiamo per misurare il modulo di recognition e per fissare la soglia. Valutare ogni modulo sul suo benchmark di riferimento è prassi normale.

Risultati sul benchmark (LFW).

Compito	Risultato
Verification 1:1 (EER)	3,30% (AUC 0,974)
Identification 1:N (rank-1)	96,77% (rank-5 97,80%)
Soglia di accettazione (cosine)	0,392

Il test sul tuo scenario (open-set). Iscriviamo 2 soggetti reali (1 foto a testa) e proviamo a: (a) riconoscere l’altra foto di ciascuno (*genuini*); (b) rifiutare degli *impostori* (foto personali + un campione di LFW, garantiti diversi).

Esito	Conteggio
Genuini identificati correttamente	2 / 2
Impostori (tuoi) rifiutati	4 / 4
Impostori (LFW) rifiutati	50 / 50
Falsi accessi	0

Perché questo step

Perché “open-set” è lo scenario forte: un sistema da poco è *costretto* a scegliere un nome (closed-set). Qui invece il sistema può dire “**SCONOSCIUTO**” quando la similarità col miglior iscritto resta sotto la soglia → rifiuta correttamente chi non è in anagrafica. È ciò che serve nella realtà (controllo accessi).

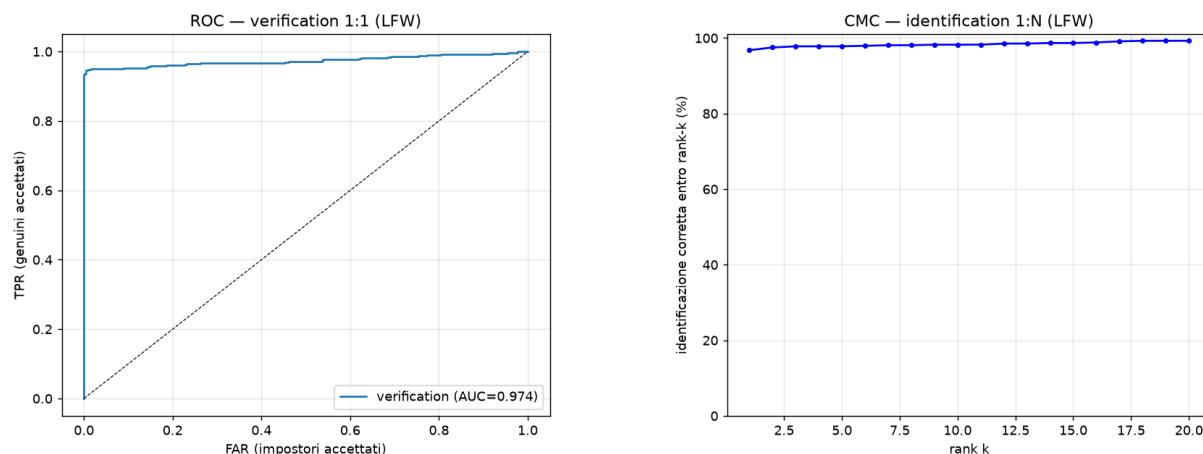


Figure 6: Sinistra: ROC della verification 1:1 su LFW. Destra: curva CMC dell'identification (rank-1 ~97%).

Nota onesta: l'anti-spoof e il domain gap (ancora)

Nel test, alcune foto-impostore reali (es. immagini WhatsApp molto compresse) sono state segnalate come **SPOOF** pur essendo persone vere. È un falso allarme dell'anti-spoof: la compressione JPEG crea artefatti che il modello, allenato su CelebA-Spoof, scambia per un attacco — di nuovo il **domain gap** della Fase 7. Qui non causa danni (l'impostore viene comunque rifiutato), ma è un limite reale da dichiarare.

14 Risultati in sintesi

	EER	AUC
Baseline SVM (LBP)	7,69%	0,976
CNN ResNet-18	3,69%	0,994
Fusione (somma)	3,35%	0,991
CNN intra-dataset (HTER)	7,91%	
CNN cross-dataset (HTER)	35,36%	

La storia in tre frasi: (1) il deep learning dimezza l'errore del metodo classico; (2) la fusione aggiunge un piccolo guadagno; (3) ma tutti i modelli soffrono il **domain gap** cross-dataset — ed è là che si gioca la vera ricerca.

15 Come spiegarlo all'orale (cheat-sheet)

- **Cos'è:** un classificatore live vs spoof per difendere il riconoscimento facciale dagli attacchi di presentazione (foto, schermo, stampa).
- **Due approcci:** feature di texture fatte a mano (LBP) + SVM *vs* CNN che impara le feature da sola. La CNN vince (EER 3,7% vs 7,7%).
- **Metriche:** EER (punto FAR=FRR), AUC, e le ISO APCER/BPCER/ACER. Più basso l'errore, più alta l'AUC.
- **Il punto forte:** valutazione **cross-dataset** (CASIA): le prestazioni crollano (HTER 8% → 35%) → il **domain gap** è il vero problema della biometria.



Figure 7: Sistema completo: a sinistra un genuino (LIVE + IDENTIFICATO), a destra un impostore (rifiutato).

- **Fusione:** multi-biometria score-level; normalizzo gli score e li sommo; piccolo guadagno; il peso “tunato” overfitta → meglio la regola semplice.
- **Demo:** funziona in tempo reale con la webcam; la calibrazione applica in pratica la lezione del domain gap.
- **Sistema completo (Fase 11):** aggiungo il riconoscimento (embedding facenet + cosine), pipeline anti-spoof → identificazione **open-set** (può dire “sconosciuto”). Su LFW: verification EER 3,3%, identification rank-1 97%. Sul mio test: 2/2 genuini identificati, 54/54 impostori rifiutati, 0 falsi accessi.

A Appendice A — Gli script (cosa fa ogni file)

Tutti i percorsi sono relativi alla cartella `Project - Face Anti-Spoofing/`. Il codice “di libreria” sta in `src/`; gli eseguibili della demo in `demo/`.

File	Fase	Cosa fa
<i>src/ — libreria</i>		
<code>src/extract.py</code>	2	Estrae i parquet di CelebA-Spoof in PNG su disco + CSV di indice. Risolve l’OOM e scarta le righe corrotte.
<code>src/data.py</code>	2	<code>Dataset</code> PyTorch che legge i PNG da disco; <code>train_val_split</code> (15% val); <code>load_csv</code> . Contiene le note anti-leakage.
<code>src/transforms.py</code>	3	Augmentation per il training e transform per la valutazione; classe <code>RandomJPEG</code> ; <code>denormalize</code> per visualizzare.
<code>src/features_classic.py</code>	4	Feature color-texture LBP (YCbCr+HSV, 2 scale → 168 numeri); estrazione parallela su un <code>DataFrame</code> .
<code>src/metrics.py</code>	4-8	Tutte le metriche: EER, FAR/FRR, APCER/BPCER/ACER (ISO), HTER, e i plot ROC/DET. Usato da tutte le fasi.
<code>src/train_svm.py</code>	4	Baseline: estrae le feature, fit SVM RBF su subset bilanciato, valuta sul test, salva modello + score.

File	Fase	Cosa fa
<code>src/model.py</code>	5	<code>build_model</code> (backbone timm, testa 2 classi) e <code>predict_scores</code> (softmax della classe spoof).
<code>src/train_cnn.py</code>	5	Training della CNN: AMP, class weights, early stopping su val EER, scheduler cosine. Salva <code>cnn_best.pt</code> + score test.
<code>src/evaluate.py</code>	6	Carica gli score salvati, tabella riassuntiva (<code>summary.csv</code>), ROC/DET di confronto; sanity check delle metriche.
<code>src/cross_dataset.py</code>	7	Valutazione intra (CelebA) vs cross (CASIA) con soglia fissata sul val sorgente; HTER e gap.
<code>src/fusion.py</code>	8	Fusione score-level SVM+CNN: normalizzazione min-max (dal val) + regole (sum, mean, product...).
<code>src/infer.py</code>	9	<code>SpoofDetector</code> : carica la CNN, face detection Haar, <code>predict/annotate</code> . Cuore della demo.
<code>src/recognition.py</code>	11	<code>FaceEmbedder</code> : MTCNN (rileva+allinea) + InceptionResnetV1 → embedding 512-d; cosine similarity.
<code>src/recognition_eval.py</code>	11	Valutazione su LFW: verification 1:1 (EER) e identification 1:N (rank-1/CMC); calibra la soglia.
<code>src/biometric_system.py</code>	11	<code>BiometricSystem</code> : anti-spoof + riconoscimento in cascata; <code>enroll</code> + <code>identify</code> open-set.
<code>src/openset_eval.py</code>	11	Test open-set sulla gallery utente: genuini vs impostori (tuoi + LFW); conta i falsi accessi.
<i>demo/ — eseguibili</i>		
<code>demo/single_image.py</code>	9	Una immagine → verdetto LIVE/SPOOF + score + immagine annotata salvata.
<code>demo/webcam_demo.py</code>	9	Webcam in tempo reale: riquadro verde REAL / rosso SPOOF + FPS.
<code>demo/calibrate.py</code>	9	Cattura tuoi frame (live/spoof) e fa un mini fine-tuning → <code>cnn_calibrated.pt</code> .
<code>demo/identify_image.py</code>	11	Sistema completo su una immagine: live/spoof + identità, immagine annotata.
<code>demo/identify_webcam.py</code>	11	Sistema completo in tempo reale: “Benvenuto, <nome>” / SPOOF / SCONOSCIUTO.

Ordine di esecuzione (entry-point principali)

1. `python src/extract.py` (Fase 2: parquet → PNG + CSV)
2. `python src/train_svm.py` (Fase 4: baseline)
3. `python src/train_cnn.py` (Fase 5: CNN)
4. `python src/evaluate.py` (Fase 6: confronto)
5. `python src/cross_dataset.py` (Fase 7: cross-dataset)
6. `python src/fusion.py` (Fase 8: fusione)
7. `python demo/webcam_demo.py` (Fase 9: demo)
8. `python src/recognition_eval.py` (Fase 11: verification/identification su LFW)
9. `python src/openset_eval.py` (Fase 11: test open-set sulla gallery)
10. `python demo/identify_webcam.py` (Fase 11: demo sistema completo)

Nota: con Python 3.14 gli script con `num_workers>0` vanno lanciati come file `.py` reali (il DataLoader usa forkserver).

B Appendice B — I dati usati (filepath)

Percorsi relativi a `Project - Face Anti-Spoofing/`. La cartella `data/` non è nel repository git (troppo grande, è in `.gitignore`).

Percorso	Dim.	Cos'è / a cosa serve
<i>Dati effettivamente usati</i>		
data/images/	12 GB	Le 145.970 immagini PNG (train+test) estratte, organizzate in <code><split>/<live spoof>/</code> . Input di training e valutazione.
data/train.csv	79.183 righe	Indice del train: colonne <code>filepath,label</code> (0=live, 1=spoof).
data/test.csv	66.787 righe	Indice del test ufficiale disgiunto (stessa struttura).
data/casia_fasd/	9,3 MB	CASIA-FASD croppato (1655 volti): il dataset “mai visto” del cross-dataset (Fase 7).
data/casia.csv	1.655 righe	Indice di CASIA-FASD (<code>filepath,label</code>).
data/gallery/<nome>/	2 sogg.	Foto dei soggetti da iscrivere (Fase 11): una per l'enrollment, le altre come probe genuini.
data/impostors/	4 foto	Tue foto di persone NON iscritte, per testare il rifiuto (Fase 11).
LFW (via <code>scikit-learn</code>)	~200 MB	Benchmark di face recognition scaricato in automatico (cache <code>~/scikit_learn_data</code>); per verification/identification e per la soglia.
<i>Sorgenti scaricate (materiale grezzo)</i>		
data/celeba_spoof_hf/	7,0 GB	Parquet del train CelebA-Spoof scaricati da Hugging Face (<code>nguyengkhoa/antispoofing</code>). Input di <code>extract.py</code> .
data/celeba_spoof_test_hf/	4,7 GB	Parquet del test CelebA-Spoof (<code>nguyengkhoa/celeba-spoof-...-test</code>).
data/casia_bahareh/	6,6 MB	Sorgente CASIA croppato (Bahareh0281/CASIA-FASD-Cropped): <code>zip + labels.csv</code> .
data/casia_fasd_hf/	64 KB	Parquet CASIA (sorgente alternativa).
data/RealDataset (mine)/	16 MB	Cartella per tue immagini personali (non usata nelle valutazioni ufficiali).

Output prodotti (cartella outputs/)

Modelli: `svm_baseline.joblib`, `cnn_best.pt`. Score salvati (per la fusione): `scores_svm_test.npz`, `scores_cnn_test.npz`.

Tabelle: `summary.csv` (Fase 6), `cross_dataset.csv` (Fase 7), `fusion.csv` (Fase 8). Grafici: `roc_*.png`, `det_*.png`, `train_curve.png`, `roc_cross_casia.png`, `roc_fusion.png`, `aug_preview.png`, `crops_preview.png`, `demo_*.png`.

Fase 11: `recognition_eval.csv`, `openset_results.csv`, `recog_threshold.npy`, `roc_verification.png`, `cmc_identification.png`, `identify_*.png`.